QUANTUM ORACLE FORENSICS

QryptoNox
RISK INTELLIGENCE

Identifying the Hidden Structure of Sealed Quantum Black Boxes | IBM Qiskit Fall Fest 2026 | QC@IITI · IIT Indore | Industry challenge by QryptoNox

1. Background

Security teams often meet cryptographic components they can only call, never read: a vendor module, a hardware device, a legacy routine. Whether such a black box hides structure that a quantum computer could exploit (a hidden period, a linear mask, a single special input) can only be probed through queries, and queries are limited and noisy. Famous quantum algorithms each solve one narrow case, but only when you already know which structure to look for. Here you do not. Design a quantum query strategy that works out what kind of black box you are facing and recovers its secret with as few queries as possible. The emphasis is on experiment design, statistics under noise, and honest validation, not on reproducing one textbook algorithm.

2. Challenge objective

Build a solver that receives one sealed oracle and a query budget, decides which of eight families the oracle belongs to, recovers its secret, and answers. Hidden oracles will test generalization. No strategy, circuit, decision threshold or sample budget split will be supplied. All oracles are synthetic; no client system or QryptoNox risk model is involved.

3. Mandatory oracle model

Every oracle takes n = 8 input bits. It is the unitary $U|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$, where f outputs m = 1 or m = 8 bits. Your circuit places the oracle gate on any qubits (the n input qubits first, then the m output qubits) and may use any other gates and extra qubits, up to 20 qubits in total. Input x is read with qubit k as bit k. The solver is told n, m, the budget and the list of family names, nothing more.

Family	m	Definition of f	Secret to report
constant	1	f(x) = c for every x	c (0 or 1)
balanced	1	random function with exactly 128 ones	none
linear	1	$f(x) = a \cdot x \oplus b$, $a \neq 0$ ($a \cdot x$ = parity of the bits shared by a and x)	[a, b], a as an integer 1–255
grover	1	f(x) = 1 for exactly one hidden input t, otherwise 0	t (0–255)
randfun	1	each f(x) = 1 independently with probability 0.3	none
simon	8	$f(x) = f(x \oplus s)$ for one hidden $s \neq 0$; otherwise all values distinct	s (1–255)
simon2	8	f is constant on the cosets of a hidden 2-dimensional subspace H and distinct otherwise (4-to-1)	the three nonzero elements of H, ascending
perm	8	random permutation of the 256 eight-bit values	none

Noise. Each oracle application misfires with probability 4% (it then acts as the identity), and each measured bit flips with probability 2%. **Budget.** 60 queries per oracle. One query is one oracle gate applied in one shot, so a circuit with k oracle gates run for s shots costs $k \times s$. Measurements must come at the end of a circuit; you may run as many circuits, one after another and adaptively, as the budget allows.

4. Public reference examples

No code, simulator or data is supplied: this document is the whole specification, and you must build your own oracle simulator and test oracles from section 3. Any function with the stated property is a valid instance. Use these examples to check your simulator (x and outputs are integers; bit k of x is qubit k):

Family	Example instance	Check values
linear	a = 178, b = 1	f(0)=1, f(1)=1, f(2)=0, f(255)=1
simon	s = 105, $f(x) = \min(x, x \oplus s)$	f(5)=5, f(108)=5, f(105)=0, f(200)=161
simon2	H = {0, 15, 49, 62}, f(x) = min over h in H of $(x \oplus h)$	f(0)=f(15)=f(49)=f(62)=0, f(1)=1, f(200)=199
perm	$f(x) = (37x + 11) \bmod 256$	f(0)=11, f(1)=48, f(2)=85, f(255)=230
grover	t = 173	f(173)=1, f(x)=0 otherwise
constant	c = 1	f(x)=1 for all x

5. Official scoring of a task

A task is correct only if the reported family is right and, for families that have a secret, the secret is exactly right. For families without a secret (balanced, randfun, perm) the family alone decides. The judges count the queries you use; unused budget is a saving. A wrong or missing answer earns nothing for that task, and exceeding the budget ends the task with no answer.

6. Required executable pipeline

Submit **solver.py** defining **solve(n, m, budget, families, run)**, which returns **(family, secret)** (secret is None for balanced, randfun and perm). **run(circuit, shots)** is supplied by the judges: it takes a Qiskit 2.x QuantumCircuit and returns a counts dictionary keyed by classical-bit strings in Qiskit order. Mark each oracle call in your circuit with a placeholder instruction **Gate("oracle", n+m, [])** acting on the n input qubits followed by the m output qubits; the judges substitute the sealed oracle. Build your own local run() from section 3 to test. Do not read, import or modify anything outside your own code.

Limits: Python 3.10+, Qiskit 2.x, numpy, scipy only (plus standard library); at most 20 qubits per circuit; 60 queries per oracle; 120 seconds per oracle; CPU only, no network, 8 GB memory. Attempting to recover an oracle by any route other than run() is disqualifying.

7. Hidden suites and timeline (24 hours)

Hidden suites keep the same eight families, n = 8 and budget 60, with unseen oracles. **Hour 0:** this document is released. **Hour 24:** tagged repository and report submitted. **After the event, Stage 1 (IITI):** runs every submission on hidden suite H (48 oracles, six per family). **Stage 2 (QryptoNox):** reruns the shortlisted solvers, unmodified, on hidden suite S (96 oracles) and on a stress setting, and reviews code and reports.

8. Quality measures

Accuracy = correct tasks / all tasks. **Worst-family accuracy** = the lowest accuracy over the eight families. **Query saving** = mean of (1 − queries used / 60) over correct tasks only. **Stress accuracy** = accuracy when the noise is doubled (8% oracle misfire, 4% readout flip). Marks rise linearly between a floor (the organizers' naive baseline) and a ceiling (the organizers' reference solver); both are calibrated by the organizers and frozen before evaluation. Guessing one family for every oracle, or applying one textbook algorithm to everything, will score poorly by design.

9. Required baselines and diagnostics

Compare on your own test oracles: (1) a classical-only strategy that queries f at chosen inputs and never uses superposition; (2) a single-algorithm strategy (for example always running the same circuit); (3) random guessing; (4) your strategy with adaptivity switched off; (5) a confusion matrix over the eight families; (6) accuracy as a function of budget (cap your own use at 20, 30, 40, 50 and 60 queries). Report honestly where a classical strategy is enough. Quantum advantage is not required at 8 bits, but explain where it does help, such as locating a single marked input with fewer than the roughly 128 expected classical queries.

10. Mathematical integrity and AI use

AI assistants and general-purpose libraries are permitted; declare assistance and independently verify results. Event-specific strategy and code must be developed during the hackathon. Decisions must come from query results only: hard-coded answers, seed lookups, or reading oracle tables are prohibited. Team size: 1–4. There is no oral defence; judging uses only submitted code, logs and the written report. Teams keep ownership of their work; QryptoNox may invite strong teams to discuss further work, and any use of code needs separate written terms.

11. Optional extensions

Examples: sequential tests with stated error guarantees and early stopping, additional families (a hidden shift of a bent function, a 3-dimensional hidden subspace, noisy Bernstein–Vazirani), error mitigation for oracle misfires, efficient simulation at larger n, or hardware-oriented analysis of one sub-protocol on a 156-qubit IBM-class layout. Extensions require a precise definition and reproducible evidence; scale alone earns no credit.

12. Deliverables

Submit a public GitHub repository containing solver.py, requirements, instructions and logs of your own tests; a technical report of at most three pages explaining your strategy, the statistics and thresholds behind each decision, how you split the budget, your baselines and where your solver fails; and a one-page AI-use disclosure outside the report limit. An optional video (up to three minutes) may supplement the submission but is not scored. No interviews or oral presentations.

13. Evaluation and rubric (100 marks)

Stage 1, IITI: runs the official harness on every submission with hidden suite H, applies the automatic items below, and forwards a fixed shortlist (size announced before the event) to QryptoNox. **Stage 2, QryptoNox:** reruns shortlisted solvers on suite S and the stress setting, reviews strategy, code and reports, and sends scores and a ranked list to the judging panel, which decides the winner.

Criterion	Marks	Assessment	By
Identification accuracy	35	Accuracy on hidden suites H and S	IITI (H); QryptoNox (S)
Query efficiency	15	Query saving on correct tasks	IITI (H); QryptoNox (S)
Robustness	15	Worst-family accuracy (8); stress-noise accuracy (7)	IITI (H); QryptoNox (S, stress)
Strategy and statistical reasoning	20	Report: decision rules, thresholds, budget allocation, baselines, failure analysis	QryptoNox
Code quality and reproducibility	10	Clean, runs unmodified, test logs match the report	IITI (5); QryptoNox (5)
Innovation and extensions	5	Relevance, definition, implementation and evidence	QryptoNox

Judging is entirely submission-based. Organizers rerun the tagged code on the official harness and compare official results with the report. Claims the logs do not support cannot earn full credit. The prize is a single ₹15,000 award to the top-ranked team. Performance thresholds and judging hardware are frozen before evaluation.